KJO Liquid Rocket Motor Control System

System Architecture and Operational Overview

Ken Overton

KJO Liquid Rocket Motor Control System

Version: 0.1
Date: March 2026

This document provides a high-level structural and functional overview of the KJO control system for a bi-propellant liquid rocket motor. It describes the three electronic subsystems (RCU, EMU, GSEMU), the propellant flow architecture, the inter-unit communication protocols, and the complete command set. Detailed subsystem reference documents are listed in Section 6.

© 2025–2026 Ken Overton. All rights reserved.

Contents

1 Introduction **2**

2 Propellant System **2**
2.1 Flow Architecture . . . 2
2.2 Fill Sequence . . . 2
2.3 Engine Firing Sequence . . . 2
2.4 Dump / Safe Procedure . . . 2
2.5 Pressurization . . . 3

3 Electronic Control Architecture **3**
3.1 RCU–EMU Radio Link . . . 3
3.2 EMU–GSEMU GPIO Handshake . . . 3

4 Shared Libraries **4**

5 Command Set **4**
5.1 Packet Structures . . . 4
5.2 Command Table . . . 4

6 Document References **5**

7 Version History **5**

1 Introduction

The KJO system controls a bi-propellant liquid rocket motor using nitrous oxide (N_2O) as the oxidizer and a liquid fuel with regenerative cooling. Three autonomous microcontroller units share control responsibilities:

- **Remote Control Unit (RCU)** — handheld operator terminal. Sends commands over a 900 MHz LoRa radio link; displays system status on a 3.5" colour touchscreen.
- **Engine Management Unit (EMU)** — mounted in the rocket. Receives commands from the RCU, actuates the Oxidizer Feed, Fuel Feed, and Dump servos, reads all sensors, and coordinates the Fill valve with the GSEMU.
- **GSE Management Unit (GSEMU)** — mounted on the launch pad. Controls the Fill valve servo and the umbilical quick-release servo; communicates valve status back to the EMU over a two-wire GPIO handshake.

All three units are based on the Adafruit Feather M0 (ATSAMD21G18A, 48 MHz ARM Cortex-M0+, 3.3 V logic) and run firmware written in C++17 using the Arduino / PlatformIO framework.

2 Propellant System

2.1 Flow Architecture

Figure **??** shows the propellant system schematic.

2.2 Fill Sequence

1. Operator commands `BEGIN_FILL` from the RCU.
2. EMU asserts the CMD bit (AUX_IO_1) high.
3. GSEMU detects the CMD rising edge and opens the Fill valve servo.
4. N_2O flows: GSE source tank → Fill valve → QR servo → umbilical → check valve → on-board oxidizer tank.
5. GSEMU asserts STATUS (AUX_IO_2) high when fill is complete.
6. EMU de-asserts CMD; GSEMU closes the Fill valve.

2.3 Engine Firing Sequence

1. Operator commands `ENABLE_LCO_CONTROL` (enables launch authority).
2. Operator commands `START_ENGINE`, which calls `Execute_Command(START_ENGINE)`.
3. EMU simultaneously opens Ox Feed valve and Fuel Feed valve to programmed positions and activates the igniter relay.
4. Proportional control commands (`OXIDIZER_FLOW_PROP`, `FUEL_FLOW_PROP`) may be sent during burn to adjust mixture ratio.
5. `STOP_ENGINE_FLOWS` closes both feed valves to terminate thrust.

2.4 Dump / Safe Procedure

The `DUMP` command opens the Dump valve (servo, EMU-controlled), venting the on-board N_2O tank to atmosphere via a vent line. The RCU Dump button has latching behaviour — a second press sends `STOP_DUMP` and closes the valve.

2.5 Pressurization

The on-board N_2O tank is self-pressurizing (autogenous). The vapour-space pressure is communicated to the fuel tank via a dedicated pressurization line, eliminating the need for a separate pressurant.

3 Electronic Control Architecture

Figure 2 shows the three-unit electronic control architecture.

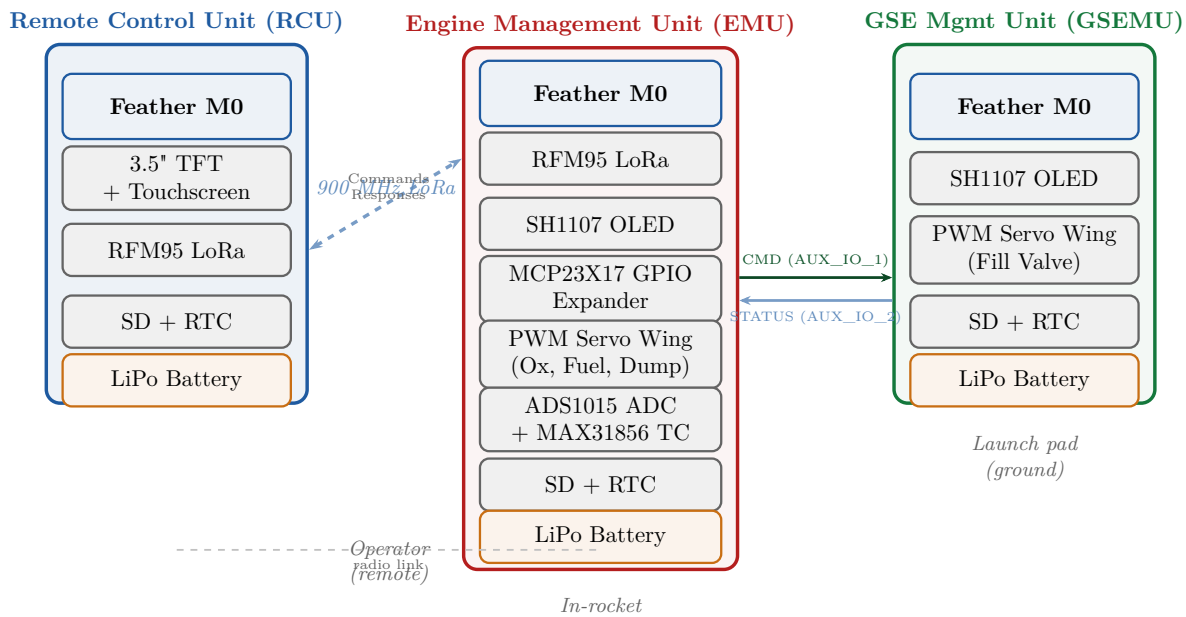

Figure 2: Electronic control architecture. The RCU communicates with the EMU over a 900 MHz LoRa radio link. The EMU communicates with the GSEMU over a two-wire GPIO handshake (CMD and STATUS).

3.1 RCU–EMU Radio Link

The link uses the HopeRF RFM95W module (Semtech SX1276 chipset) operating at 915.0 MHz with 23 dBm transmit power. Packets are simple binary structures (see Section **??**). The RCU sends a **`HEARTBEAT`** packet at 5-second intervals when no other command is pending; the EMU echoes a **`HEARTBEAT`** response containing valve positions and battery voltage. Link loss is declared after 30 seconds without a response.

3.2 EMU–GSEMU GPIO Handshake

Two digital lines on the MCP23X17 GPIO expander carry the Fill valve handshake:

- `AUX_IO_1` (CMD): driven by the EMU; asserted high to command the Fill valve open, de-asserted to close.
- `AUX_IO_2` (STATUS): driven by the GSEMU; asserted high when the Fill valve has reached its commanded position.

Additional GPIO lines (`AUX_IO_3`, `AUX_IO_4`) are reserved for the umbilical-release handshake (future implementation).

4 Shared Libraries

Common definitions reside in `KJO_Shared_Libraries` and are shared across all three firmware projects via PlatformIO library dependencies:

Library	Contents	Used by
`KJO_System_Config`	Platform IDs, version strings	RCU, EMU, GSEMU
`KJO_Command_Defs`	Command codes, packet structs	RCU, EMU
`KJO_Radio`	Radio pin assignments, frequency	RCU, EMU
`KJO_RCU_Display`	TFT GUI functions and data types	RCU

5 Command Set

5.1 Packet Structures

Three binary packet types are used on the radio link:

- **Command_Payload** (4 bytes): `command` (2B), `param` (short, 2B). Used for most commands.
- **Heartbeat_Command_Packet** (6 bytes): `command` (2B), `RCU_voltage` (float, 4B). Sent by RCU as heartbeat.
- **Heartbeat_Response_Packet** (10 bytes): `command` (2B), `EMU_voltage` (float, 4B), `fill_pct`, `ox_pct`, `fuel_pct`, `dump_pct` (uint8, 1B each). Sent by EMU.
- **Command_Return_Packet**: `command` (2B), `status` (bool), `r_val` (short). Returned for non-heartbeat commands.

5.2 Command Table

Code	Name	Direction	`param`	Description
1	`BEGIN_FILL`	RCU→EMU	—	Command GSEMU to open Fill valve
2	`STOP_FILL`	RCU→EMU	—	Command GSEMU to close Fill valve
3	`OPEN_OXIDIZER_FEED`	RCU→EMU	—	Open Ox Feed valve fully
4	`CLOSE_OXIDIZER_FEED`	RCU→EMU	—	Close Ox Feed valve
5	`OPEN_FUEL_FEED`	RCU→EMU	—	Open Fuel Feed valve fully
6	`CLOSE_FUEL_FEED`	RCU→EMU	—	Close Fuel Feed valve
7	`OXIDIZER_FLOW_PROP`	RCU→EMU	%	Set Ox valve position (0–100%)
8	`FUEL_FLOW_PROP`	RCU→EMU	%	Set Fuel valve position (0–100%)
9	`DUMP`	RCU→EMU	—	Open Dump valve; vent oxidizer
10	`STOP_DUMP`	RCU→EMU	—	Close Dump valve
11	`START_ENGINE_FLOWS`	RCU→EMU	—	Open both feed valves simultaneously
12	`STOP_ENGINE_FLOWS`	RCU→EMU	—	Close both feed valves simultaneously
13	`ENABLE_LCO_CONTROL`	RCU→EMU	—	Enable launch authority
14	`START_ENGINE`	RCU→EMU	—	Fire igniter and open valves
15	`RESET`	RCU→EMU	—	Reset EMU to safe state

Code	Name	Direction	param	Description
16	`READ_TANK_PRESSURE`	RCU→EMU	—	Return oxidizer tank pressure
17	`READ_CHAMBER_PRESSURE`	RCU→EMU	—	Return chamber pressure
18	`READ_TANK_TEMP`	RCU→EMU	—	Return oxidizer tank temperature
19	`READ_CHAMBER_TEMP`	RCU→EMU	—	Return combustion chamber temperature
20	`START_PRESSURE_RECORDING`	RCU→EMU	—	Begin high-rate pressure logging
21	`STOP_PRESSURE_RECORDING`	RCU→EMU	—	End high-rate pressure logging
22	`READ_VALVE_POSITION`	RCU→EMU	valve#	Return servo position (0–100%)
23	`HEARTBEAT`	Bidirectional	—	Keepalive / link maintenance
24	`RCU_STATUS`	RCU→EMU	—	Request full status packet

6 Document References

Document	File	Contents
RCU Reference	`RCU/KJO_RCU_Reference.pdf`	Hardware, software, GUI
EMU Reference	`EMU/KJO_EMU_Reference.pdf`	Hardware, software, valve control
GSEMU Reference	`GSEMU/KJO_GSEMU_Reference.pdf`	Hardware, software, Fill valve

7 Version History

Version	Date	Notes
0.1	March 2026	Initial release.